Практика №2. Аудит готового клиент-серверного приложения

1) Ответы на вопросы

Вопрос 1. Это TCP. Тип сокета SOCK_STREAM и вызов connect() как раз про TCP: он сначала подключается к серверу, а потом уже отправляет данные.

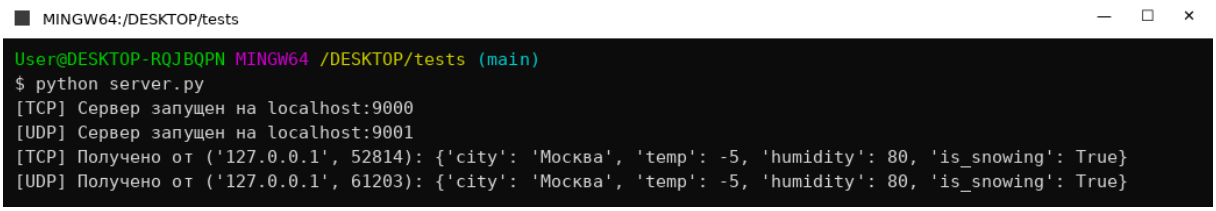
Вопрос 2. Это UDP. SOCK_DGRAM означает отдельные пакеты, а sendto() шлёт пакет сразу, без подключения. Что он дойдёт, никто не гарантирует, поэтому в коде и стоит ожидание ответа 2 секунды.

Вопрос 3. JSON. В client.py есть строка `client.send(json.dumps(message, ensure_ascii=False).encode("utf-8"))`. Здесь json.dumps превращает словарь weather в текст в формате JSON, а encode переводит его в байты. Сервер потом читает всё это через json.loads.

Вопрос 4. Сервер попробует разобрать "hello" как JSON, но не сможет, и будет ошибка JSONDecodeError. Она ловится в except, поэтому сервер не падает: он пишет в консоль, что не удалось разобрать JSON, и отправляет клиенту {"status": "ERROR", "reason": "invalid JSON"}.

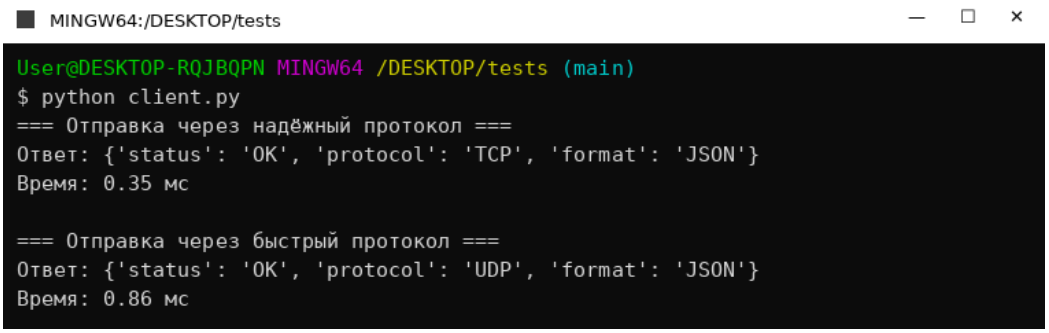
2) Запуск сервера и клиента

Окно сервера



Сервер запустился на портах 9000 (TCP) и 9001 (UDP). После запуска клиента в окне появились две строки «Получено», одна от TCP и одна от UDP, в обеих данные о погоде в Москве.

Окно клиента



Клиент получил два ответа со статусом OK. По TCP вышло 0,35 мс, по UDP 0,86 мс, то есть на одном компьютере разницы почти нет.

3) Тестирование

Случай 1. Сервер выключен, клиент TCP

Клиент сразу упал с ошибкой ConnectionRefusedError (Connection refused). TCP сначала подключается к серверу, а сервера нет и подключаться не к чему. До UDP-части дело вообще не дошло, потому что программа остановилась на первой ошибке.

Случай 2. Сервер выключен, клиент UDP

Ошибки не было. Клиент подождал 2 секунды (в консоли «Время: 2002.67 мс») и вывел {'status': 'TIMEOUT'}. UDP не проверяет, дошло ли сообщение, поэтому клиент не понял, что сервера нет, и просто перестал ждать по таймеру.

Случай 3. Клиент отправляет не JSON

Я заменил строку отправки на `client.send(b"hello")`. Сервер ответил {'status': 'ERROR', 'reason': 'invalid JSON'}, а у себя в консоли написал «[TCP] Ошибка: не удалось разобрать JSON». Так вышло потому, что hello не JSON и json.loads выдал ошибку. Потом я вернул строку обратно.

Случай 4. Сравнение форматов

JSON получился 72 байта, XML 111 байт, так что JSON компактнее. В XML у каждого поля два тега (открывающий и закрывающий), а в JSON имя поля пишется один раз. Ещё я заметил, что из XML все значения восстанавливаются текстом ('-5', 'True'), а из JSON приходят с нормальными типами.

4) Вывод

Я бы выбрал TCP и JSON.

TCP надёжнее: данные точно дойдут, а если сервера нет, сразу видна ошибка. С UDP клиент просто ждал 2 секунды и было непонятно, что случилось. По скорости я разницы не увидел. JSON выбрал, потому что он короче XML и сохраняет типы данных.